

VLLM专题（四十一）—vLLM开发/分析与构建



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

1. 开发

根据您希望进行的开发类型（例如Python、CUDA），您可以选择是否在编译的情况下构建vLLM。详情请查看从源代码构建的文档。

2. 测试

```
pip install -r requirements/dev.txt

# Linting, formatting and static type checking
pre-commit install --hook-type pre-commit --hook-type commit-msg

# You can manually run pre-commit with
pre-commit run --all-files

# Unit tests
pytest tests/
```

注意：

目前，`mypy` 尚未对仓库进行全面的类型检查。

3. 问题

如果您遇到错误或有功能请求，请首先搜索现有问题，看看是否已经有人报告过。如果没有，请提交一个新问题，并提供尽可能多的相关信息。

4. 拉取请求与代码审查

感谢您对vLLM的贡献！在提交拉取请求之前，请确保PR符合以下标准。这有助于vLLM保持代码质量并提高审查过程的效率。

4.1 DCO与签署提交

当您向本项目贡献更改时，您必须同意开发者证书（DCO）。提交必须包含一个 `Signed-off-by` :头部，以证明您同意DCO的条款。

使用 `git commit` 时加上 `-s` 选项将自动添加此头部。

4.2 PR标题与分类

只有特定类型的PR会被审查。PR标题应适当添加前缀以指示更改的类型。请使用以下之一：

`[Bugfix]` 用于错误修复。

`[CI/Build]` 用于构建或持续集成的改进。

`[Doc]` 用于文档修复和改进。

[Model] 用于添加新模型或改进现有模型。模型名称应出现在标题中。

[Frontend] 用于vLLM前端的更改（例如，OpenAI API服务器、LLM类等）。

[Kernel] 用于影响CUDA内核或其他计算内核的更改。

[Core] 用于vLLM核心逻辑的更改（例如，LLMEngine、AsyncLLMEngine、调度程序等）。

[Hardware][Vendor] 用于特定硬件的更改。供应商名称应出现在前缀中（例如，[Hardware][AMD]）。

[Misc] 用于不符合上述类别的PR。请谨慎使用此分类。

注意

如果PR涉及多个类别，请包含所有相关前缀。

4.3 代码质量

PR需要满足以下代码质量标准：

我们遵循Google Python风格指南和Google C++风格指南。

通过所有linter检查。请使用 **pre-commit** 来格式化您的代码。如果 **pre-commit** 对您来说是新的，请参阅<https://pre-commit.com/#usage>。

代码需要良好文档化，以确保未来的贡献者能够轻松理解代码。

包含足够的测试，以确保项目保持正确和健壮。这包括单元测试和集成测试。

如果PR修改了vLLM的用户行为，请向 [docs/source/](#) 添加文档。这有助于vLLM用户理解并利用新功能或更改。

4.4 添加或更改内核

每个自定义内核都需要一个模式和一个或多个实现来注册到PyTorch。

确保自定义操作按照PyTorch指南注册：自定义C++和CUDA操作符和自定义操作符手册。

返回张量的自定义操作需要元函数。元函数应在Python中实现并注册，以便自动处理动态维度。有关元函数的描述，请参阅上述文档。

使用 `torch.library.opcheck()` 测试任何注册操作的函数注册和元函数。参见 `tests/kernels` 中的示例。

当更改现有操作的C++签名时，必须更新模式以反映更改。

4.5 大型更改的注意事项

请尽量保持更改简洁。对于主要的架构更改（>500 LOC，不包括内核/数据/配置/测试），我们希望有一个GitHub问题（RFC）讨论技术设计和理由。否则，我们将标记为rfc-required，并且可能不会通过PR。

4.6 审查过程中的期望

vLLM团队的目标是成为一个透明的审查机器。我们希望使审查过程透明且高效，并确保没有贡献者感到困惑或沮丧。然而，vLLM团队规模较小，因此我们需要优先处理一些PR。以下是您可以从审查过程中期望的：

PR提交后，PR将被分配给一个审查者。每个审查者将根据他们的专业知识和可用性来挑选PR。

PR分配后，审查者将每2-3天提供状态更新。如果PR在7天内未被审查，请随时联系审查者或vLLM团队。

审查后，如果需要更改，审查者将在PR上放置一个action-required标签。贡献者应解决评论并联系审查者重新审查PR。

请在合理的时间范围内回应所有评论。如果评论不清楚或您不同意建议，请随时要求澄清或讨论建议。

请注意，由于计算资源有限，并非所有CI检查都会执行。当PR准备好合并或需要完整的CI运行时，审查者将添加ready标签。

5. 分析 vLLM

警告

分析仅适用于vLLM开发者和维护者，以了解代码库中不同部分所花费的时间比例。vLLM的最终用户不应开启分析，因为它会显著减慢推理速度。

6. 使用PyTorch Profiler进行分析

我们支持使用torch.profiler模块跟踪vLLM工作进程。您可以通过设置 `VLLM_TORCH_PROFILER_DIR` 环境变量为保存跟踪文件的目录来启用跟踪：`VLLM_TORCH_PROFILER_DIR=/mnt/traces/`

启动OpenAI服务器时也需要设置 `VLLM_TORCH_PROFILER_DIR` 环境变量。

使用 `benchmarks/benchmark_serving.py` 时，您可以通过传递 `--profile` 标志来启用分析。

可以使用<https://ui.perfetto.dev/>可视化跟踪文件。

提示

在进行分析时，只需通过vLLM发送少量请求，因为跟踪文件可能会变得非常大。此外，无需解压跟踪文件，可以直接查看。

提示

要停止分析器 - 它会将所有分析跟踪文件刷新到目录中。这需要时间，例如对于大约100个请求的数据量（如llama 70b），在H100上大约需要10分钟来刷新。在启动服务器之前，将环境变量VLLM_RPC_TIMEOUT设置为一个较大的数值。例如，设置为30分钟：`export VLLM_RPC_TIMEOUT=1800000`。

6.1 示例命令和用法

离线推理

请参阅如下 `simple_profiling.py` 代码以获取示例。

`simple_profiling.py`

```
# SPDX-License-Identifier: Apache-2.0

import os
import time

from vllm import LLM, SamplingParams

# enable torch profiler, can also be set on cmd line
os.environ["VLLM_TORCH_PROFILER_DIR"] = "./vllm_profile"

# Sample prompts.
prompts = [
```

```

    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]
# Create a sampling params object.
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

if __name__ == "__main__":

    # Create an LLM.
    llm = LLM(model="facebook/opt-125m", tensor_parallel_size=1)

    llm.start_profile()

    # Generate texts from the prompts. The output is a list of RequestOutput
# objects that contain the prompt, generated text, and other information.
    outputs = llm.generate(prompts, sampling_params)

    llm.stop_profile()

    # Print the outputs.
    for output in outputs:
        prompt = output.prompt
        generated_text = output.outputs[0].text
        print(f"Prompt: {
            prompt!r}, Generated text: {
            generated_text!r}")

    # Add a buffer to wait for profiler in the background process
# (in case MP is on) to finish writing profiling output.
    time.sleep(10)

```

6.2 OpenAI 服务

```
VLLM_TORCH_PROFILER_DIR=./vllm_profile python -m vllm.entrypoints.openai.api_server --model meta-llama/Meta-Llama-3-70B
```

benchmark_serving.py:

```
python benchmarks/benchmark_serving.py --backend vllm --model meta-llama/Meta-Llama-3-70B --dataset-name sharegpt --dataset-path sharegpt.json --profile --num-prompts 2
```

7. 使用 NVIDIA Nsight Systems 进行分析

Nsight Systems 是一个高级工具，可以暴露更多的分析细节，例如寄存器和共享内存使用情况、注释的代码区域以及低级别的 CUDA API 和事件。

使用您的包管理器安装 nsight-systems。以下代码块是 Ubuntu 的示例。

```
apt update
apt install -y --no-install-recommends gnupg
echo "deb http://developer.download.nvidia.com/devtools/repos/ubuntu$(source /etc/lsb-release; echo "$DISTRIB_RELEASE" | tr -d .)/$(dpkg --print-architecture) /" | tee /etc/apt/sources.list.d/nvidia-devtools.list
apt-key adv --fetch-keys http://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/x86_64/7fa2af80.pub
apt update
apt install nsight-systems-cli
```

7.1 示例命令和用法

7.1.1 离线推理

对于基本用法，您只需在运行离线推理的任何现有脚本前添加 `nsys profile -o report.nsys-rep --trace-fork-before-exec=true --cuda-graph-trace=node`。

以下是使用 `benchmarks/benchmark_latency.py` 脚本的示例：

```
nsys profile -o report.nsys-rep --trace-fork-before-exec=true --cuda-graph-trace=node python benchmarks/benchmark_latency.py --model meta-llama/Llama-3.1-8B-Instruct --num-iters-warmup 5 --num-iters 1 --batch-size 16 --input-len 512 --output-len 8
```

7.1.2 OpenAI 服务

要对服务器进行分析，您需要像离线推理一样在 `vllm serve` 命令前添加 `nsys profile`，但必须根据您的基准测试需求指定 `--delay XX --duration YY` 参数。持续时间用完后，服务器将被终止。

8. 服务端

```
# server
nsys profile -o report.nsys-rep --trace-fork-before-exec=true --cuda-graph-trace=node --delay 30 --duration 60 vllm serve meta-llama/Llama-3.1-8B-Instruct

# client
python benchmarks/benchmark_serving.py --backend vllm --model meta-llama/Llama-3.1-8B-Instruct --num-prompts 1 --dataset-name random --random-input 1024 --random-output 512
```

实际上，您应将 `--duration` 参数设置为一个较大的值。当您希望服务器停止分析时，运行：

```
nsys sessions list
```

以获取形式为 `profile-XXXXXX` 的会话ID，然后运行：

```
nsys stop --session=profile-XXXXX
```

手动终止分析器并生成您的nsys-rep报告。

分析

您可以在CLI中使用`nsys stats [profile-file]`查看这些分析的摘要，或者按照此处的说明在本地安装Nsight，通过GUI查看。

CLI示例：

```
nsys stats report1.nsys-rep
```

```
...
```

```
** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	Std Dev (ns)	Name
----------	-----------------	-----------	----------	----------	----------	----------	--------------	------

46.3	10,327,352,338	17,505	589,965.9	144,383.0	27,040	3,126,460	944,263.8	sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_til... execute_segment_k_of...
14.8	3,305,114,764	5,152	641,520.7	293,408.0	287,296	2,822,716	867,124.9	sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_til... execute_segment_k_of...
12.1	2,692,284,876	14,280	188,535.4	83,904.0	19,328	2,862,237	497,999.9	sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_til... execute_segment_k_off...
9.5	2,116,600,578	33,920	62,399.8	21,504.0	15,326	2,532,285	290,954.1	sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_til... execute_segment_k_off...
5.0	1,119,749,165	18,912	59,208.4	9,056.0	6,784	2,578,366	271,581.7	void vllm::act_and_mul_kernel<c10::BFloat16, &vllm::silu_kernel<c10::BFloat16>

```

71,501.7 void vllm::act_and_mat_kernel<c10::BFloat16, __vllm::S116_kernel<c10::BFloat16>,
 (bool)1>(T1 *, cons...
      4.1      916,662,515      21,312      43,011.6      19,776.0      8,928  2,586,205  1
99,790.1 void cutlass::device_kernel<flash::enable_sm90_or_later<flash::FlashAttnFwdSm90
<flash::CollectiveMa...
      2.6      587,283,113      37,824      15,526.7      3,008.0      2,719  2,517,756  1
39,091.1 std::enable_if<T2>(int)0&&vllm::_typeConvert<T1>::exists, void>::type vllm::fus
ed_add_rms_norm_kern...
      1.9      418,362,605      18,912      22,121.5      3,871.0      3,328  2,523,870  1
75,248.2 void vllm::rotary_embedding_kernel<c10::BFloat16, (bool)1>(const long *, T1 *,
T1 *, const T1 *, in...
      0.7      167,083,069      18,880      8,849.7      2,240.0      1,471  2,499,996  1
01,436.1 void vllm::reshape_and_cache_flash_kernel<__nv_bfloat16, __nv_bfloat16, (vllm::
Fp8KVCacheDataType)0...
...

```

GUI示例:

9. Dockerfile

我们提供了一个Dockerfile，用于构建运行与vLLM兼容的OpenAI服务器的镜像。有关使用Docker部署的更多信息可以在这里找到。

以下是多阶段Dockerfile的视觉表示。构建图包含以下节点：

- 所有构建阶段
- 默认构建目标（以灰色突出显示）
- 外部镜像（带有虚线边框）

构建图的边表示：

- FROM ... 依赖项（带有实线和完整箭头）
- COPY --from=... 依赖项（带有虚线和空箭头）
- RUN --mount=(.*)from=... 依赖项（带有点线和空菱形箭头）

使用以下命令重新生成构建图（确保在 vLLM 仓库的根目录下运行，其中包含 Dockerfile）：

```
dockerfilegraph -o png --legend --dpi 200 --max-label-length 50 --filename Dockerfile
```

或者，如果你想直接使用 Docker 镜像运行：

```
docker run \
  --rm \
  --user "$(id -u):$(id -g)" \
  --workdir /workspace \
  --volume "$(pwd)":/workspace \
  ghcr.io/patrickhoefer/dockerfilegraph:alpine \
  --output png \
  --dpi 200 \
  --max-label-length 50 \
  --filename Dockerfile \
  --legend
```

（如果你想为不同的文件运行，可以将不同的文件名传递给 `--filename` 标志。）